Ummed Singh
MTS@ByteXL | Ex - Nagarro | Java | Spring Boot | Microservices | React...

#12 Entity Relationships & Mapping

Entity Relationships & Mapping	One-to-One, One-to-Many, Many-to-One, Many-to-Many mappings

Entity Relationships & Mapping in Spring Boot (JPA/Hibernate)

When working with databases, tables are often connected to each other.

Example:

- One Student belongs to one Profile
- One Department has many Students
- Many Students can enroll in many Courses

In Spring Boot, these relationships are managed using **JPA Mapping Annotations**.

Why Relationship Mapping?

Suppose we have two tables:

Student Table

ID	Name
1	Ummed
2	Rahul

Aadhar Table

ID	aadhar_number
1	63535
2	6242537

Without relationship mapping, we manually manage foreign keys.

With JPA:

Ummed Singh
MTS@ByteXL | Ex - Nagarro | Java | Spring Boot | Microservices | React...

```
student.getProfile().getAddress();
```

JPA automatically handles joins and relationships.

Types of Entity Relationships

There are 4 major mappings:

1. One-to-One
2. One-to-Many
3. Many-to-One
4. Many-to-Many

1. One-to-One Mapping

One record of Table A is associated with exactly one record of Table B.

Student <-------> Aadhar

1 Student = 1 Aasdhar_number
1 Aadhar_number = 1 Student

Real-Life Example

Person → Aadhaar Card

One Person has one Aadhaar Card.
One Aadhaar Card belongs to one Person.

Database Structure

Student

id	name
1	Ummed

Ummed Singh
MTS@ByteXL | Ex - Nagarro | Java | Spring Boot | Microservices | React...

Aadhar

id	aadhar_number	student_id
101	746675	1

Foreign Key: student_id

Entity Classes

Student Entity

```
@Entity
public class Student {

   @Id
   @GeneratedValue
   private Long id;

   private String name;

   @OneToOne
   @JoinColumn(name = "profile_id")
   private Profile profile;
}
```

Profile Entity

```
@Entity
public class Profile {

   @Id
   @GeneratedValue
   private Long id;

   private String address;
}
```

Ummed Singh
MTS@ByteXL | Ex - Nagarro | Java | Spring Boot | Microservices | React...

Generated Tables

One-to-One Flow

Student
|
| 1 : 1
|
Profile

---

2. One-to-Many Mapping

One record can be related to multiple records.

Department --> Students

One Department
Many Students

Real-Life Example

Department = CSE

Students:

- Ummed
- Rahul
- Amit

One department contains many students.

Database Structure

Ummed Singh
MTS@ByteXL | Ex - Nagarro | Java | Spring Boot | Microservices | React...

Department

dept_id	dept_name
1	CSE

Student

id	name	dept_id
1	Ummed	1
2	Rahul	1
3	Amit	1

Department Entity

```
@Entity
public class Department {

   @Id
   @GeneratedValue
   private Long id;

   private String deptName;

   @OneToMany(mappedBy = "department")
   @JsonIgnore
   private List<Student> students;
}
```

Student Entity

```
@Entity
public class Student {

   @Id
   @GeneratedValue
   private Long id;
```

```
    private String name;

    @ManyToOne
    @JoinColumn(name = "dept_id")
    private Department department;
}
```

Flow

Department
|
| 1 : N
|
Students

---

3. Many-to-One Mapping

Many records belong to one record.

Many Students
|
|
V
One Department

Real-Life Example

Ummed
Rahul
Amit
|
|
V
CSE Department

Many students belong to one department.

Ummed Singh
MTS@ByteXL | Ex - Nagarro | Java | Spring Boot | Microservices | React...

Student Entity

```
@ManyToOne
@JoinColumn(name="dept_id")
private Department department;
```

SQL Representation

Student
---
id
name
dept_id

Here: **dept_id**

acts as **Foreign Key.**

Difference Between OneToMany and ManyToOne

OneToMany View

Department Perspective

Department
  |
  ├── Student1
  ├── Student2
  └── Student3

Code:

```
@OneToMany
private List<Student> students;
```

ManyToOne View

Student Perspective

Student
  |

V
Department

Code:

```
@ManyToOne
private Department department;
```

Both represent the same relationship from different sides.

---

4. Many-to-Many Mapping

Many records can be related to many records.

Many Students
↔
Many Courses

Real-Life Example

Students

Ummed
Rahul

Courses

Java
Spring Boot
React

Relationships:

Ummed → Java
Ummed → Spring Boot

Rahul → Java
Rahul → React

Ummed Singh
MTS@ByteXL | Ex - Nagarro | Java | Spring Boot | Microservices | React...

Why Need Join Table?

- Direct FK cannot handle many-to-many.
- JPA creates a third table.

Database Structure

Student

id	name
1	Ummed
2	Rahul

Course

id	course_name
101	Java
102	Spring Boot

Student_Course

student_id	course_id
1	101
1	102
2	101

Student Entity

```
@Entity
public class Student {

   @Id
   @GeneratedValue
   private Long id;
```

```
    private String name;

    @ManyToMany
    @JoinTable(
            name = "student_course",
            joinColumns =
            @JoinColumn(name = "student_id"),
            inverseJoinColumns =
            @JoinColumn(name = "course_id")
    )
    private List<Course> courses;
}
```

Course Entity

```
@Entity
public class Course {

    @Id
    @GeneratedValue
    private Long id;

    private String courseName;
}
```

Flow

Student
↕
Student_Course
↕
Course

Example:

Ummed
├── Java
├── Spring Boot

Rahul

Ummed Singh
MTS@ByteXL | Ex - Nagarro | Java | Spring Boot | Microservices | React...

├── Java
└── React

@JoinColumn

Used to create Foreign Key.

Example:

```
@ManyToOne
@JoinColumn(name = "dept_id")
private Department department;
```

Generated SQL:

```
dept_id BIGINT
```

This column stores Department ID inside Student table.

mappedBy Attribute

Used in Bidirectional Mapping.

Example:

```
@OneToMany(mappedBy = "department")
private List<Student> students;
```

Meaning:

- Department is NOT the owner.
- Student owns the relationship.

Foreign key exists in:

Student Table

not Department table.

Ummed Singh
MTS@ByteXL | Ex - Nagarro | Java |
Spring Boot | Microservices | React...

Interview Questions

Q1. What is the difference between OneToMany and ManyToOne?

Answer:
Both represent the same relationship from different sides.

Department → Students = OneToMany

Student → Department = ManyToOne

Q2. Why is @JoinColumn used?

Answer:

To create and manage a Foreign Key column.

@JoinColumn(name="dept_id")

Q3. Why is a Join Table required in ManyToMany?

Answer:

Because one foreign key cannot store multiple values.

So JPA creates:

student_course

table.

Q4. What is mappedBy?

Answer:

It specifies the non-owning side of a relationship and prevents duplicate foreign keys.

Ummed Singh
MTS@ByteXL | Ex - Nagarro | Java | Spring Boot | Microservices | React...

```
@OneToMany(mappedBy="department")
```